BÁO CÁO BÀI TẬP LỚN 2

ĐỀ TÀI:

Vehicle Detection using Deep Learning

GVHD:

- TS Lê Thành Sách

Group 3:

- Đồng Quang Trí - 2570523
- Phạm Văn Thành - 2570496
- Bùi Minh Hiếu - 2570406
- Ngô Lê Khoa - 2570432

CONTENTS

01. Giới thiệu bài toán

02. Yolov8

03. Faster R-CNN

04. So sánh kết quả

05. Kết luận

06. Demo

01

GIỚI THIỆU BÀI TOÁN

Giới thiệu bài toán: Phát hiện phương tiện trên Pascal VOC 2007

Mục tiêu: phát hiện và định vị bốn lớp phương tiện — **car, bus, motorbike, bicycle** — trên ảnh từ tập dữ liệu Pascal VOC 2007. Bài toán yêu cầu cả độ chính xác vị trí (bounding box) và phân loại từng đối tượng trong ảnh có nền phức tạp.

Thách thức chính: biến thiên kích thước vật thể (từ nhỏ đến lớn), che khuất, góc nhìn khác nhau, và số lượng mẫu không đồng đều giữa các lớp. Yêu cầu ứng dụng: từ giám sát giao thông đến hệ thống hỗ trợ lái xe.

Phân tích dữ liệu & Tiền xử lý (EDA)

Thống kê mẫu

Tổng ảnh: ~5,011 (VOC2007 train+val/test tùy cấu hình). Phân bố lớp không đồng đều — **car** nhiều nhất, **bicycle** ít hơn. Cần cân bằng dữ liệu cho huấn luyện.

Tiền xử lý format

Chuyển nhãn Pascal XML → COCO/YOLO TXT khi cần. Chuẩn hóa kích thước ảnh (multi-scale training), chuẩn hóa ảnh theo mean/std của ImageNet cho pretrained backbone.

Augmentation

Sử dụng: Random flip, color jitter, mosaic, mixup, random crop, và multi-scale. Mosaic & Mixup giúp tăng tính đa dạng cảnh và cải thiện phát hiện vật thể nhỏ/che khuất.

Ghi chú kỹ thuật: lưu trữ thống kê bbox (kích thước, tỉ lệ) để điều chỉnh anchor/strides cho Faster R-CNN và config stride/anchor-free cho YOLOv8.

Lọc & Chuẩn bị Tập Dữ liệu (Dataset Preparation)

```
        kept_counts[split] = kept
        print(f"{split}: kept {kept} images with target classes")

    data_yaml = {
        "path": str(output_root.resolve()),
        "train": "images/train",
        "val": "images/val",
        "test": "images/test",
        "names": {i: name for i, name in enumerate(CLASS_NAMES)},
    }

    yaml_path = output_root / "data.yaml"
    with open(yaml_path, "w") as f:
        yaml.safe_dump(data_yaml, f, sort_keys=False)

    print("Saved YAML to:", yaml_path)
    return str(yaml_path), kept_counts


data_yaml_path, kept_counts = prepare_yolo_dataset(
    VOC_ROOT,
    YOLO_DATASET_ROOT,
    train_ids,
    val_ids,
    test_ids
)
```

```
train: kept 1042 images with target classes
val: kept 250 images with target classes
test: kept 1309 images with target classes
Saved YAML to: voc2007_yolo_vehicle/data.yaml
```

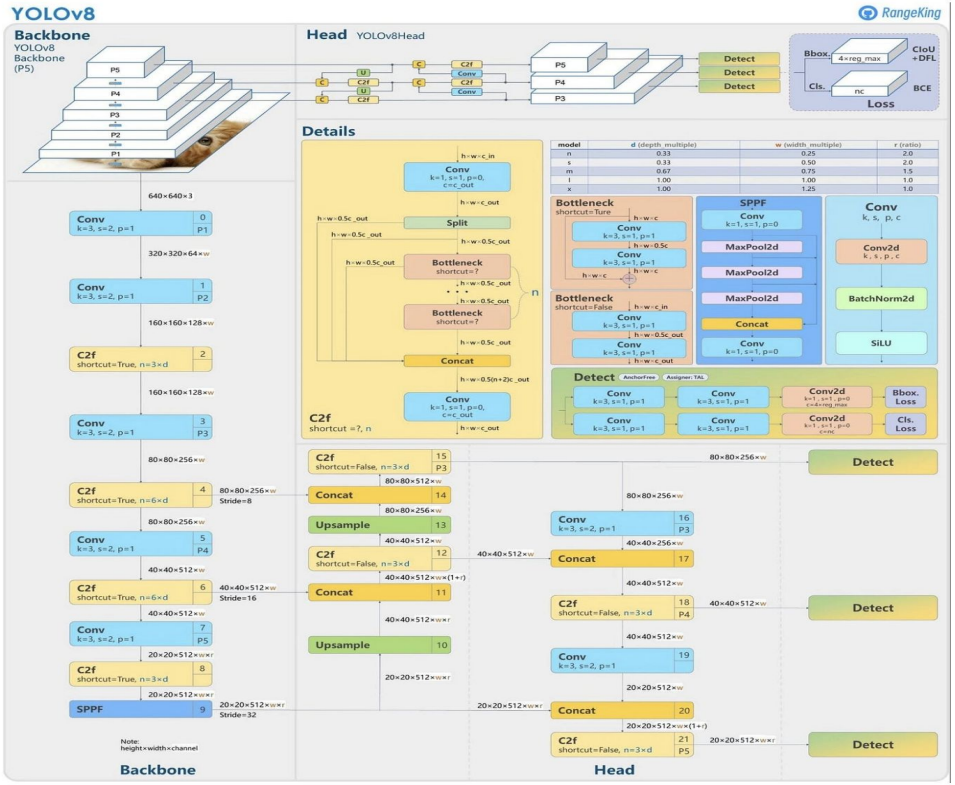
02

YOLOv8

Phương pháp 1 — YOLOv8 (One-stage)

Kiến trúc (Backbone, Neck, Head)

Backbone: CSP/Focus-like modules trích xuất đặc trưng. Neck: PAN/FPN kết hợp đa tỷ lệ. Head: head thẳng xuất bbox + class trong cùng bước—không cần RPN.



Phương pháp 1 — YOLOv8 (One-stage)

Ưu điểm

Tốc độ cao (real-time), đơn giản triển khai, tốt cho ứng dụng thời gian thực. Kiến trúc hiện đại hỗ trợ anchor-free hoặc anchor-based tùy cấu hình.

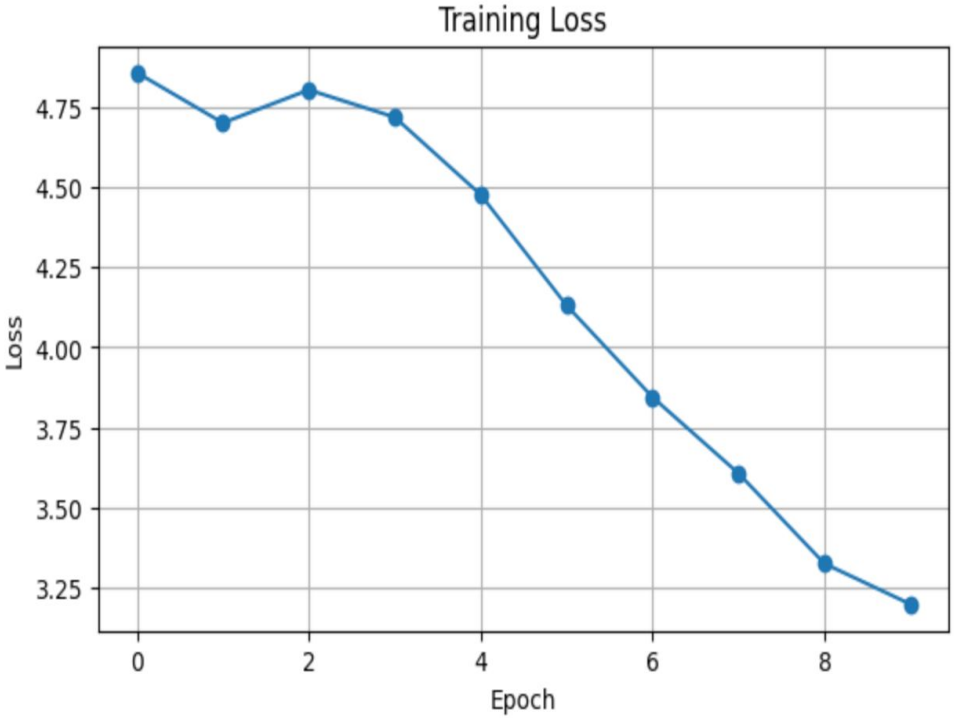
Huấn luyện & Hyper-parameters

Optimizer: SGD/AdamW - LR schedule: cosine/step, warmup (1–3 epochs). - Batch size: 16–64 (tùy GPU). - Augmentations: mosaic, mixup, multi-scale. - Epochs: 50–200 (tùy mục tiêu). Ghi chú: điều chỉnh image size & anchor/stride để cải thiện mAP cho các object nhỏ.

Đồ thị Huấn luyện - Training Loss

Xu hướng: Tổng mức lỗi (Loss) giảm đều đặn từ khoảng 4.85 xuống còn 3.2 sau 10 epochs.

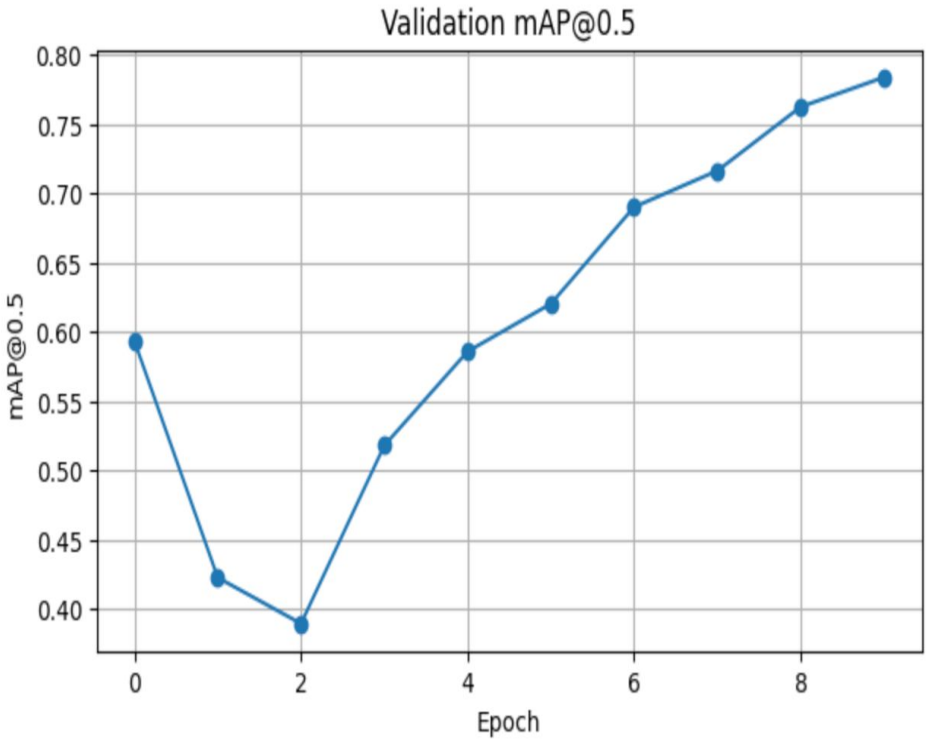
Đánh giá: Mô hình hội tụ tốt, thuật toán tối ưu (AdamW/SGD) đang hoạt động hiệu quả.



Đồ thị Đánh giá - Validation mAP@0.5

Xu hướng: Có một nhịp giảm (dip) ở 2 epoch đầu (thường do quá trình khởi động Learning Rate Warmup), sau đó tăng vọt và đạt đỉnh ~78% ở epoch 10

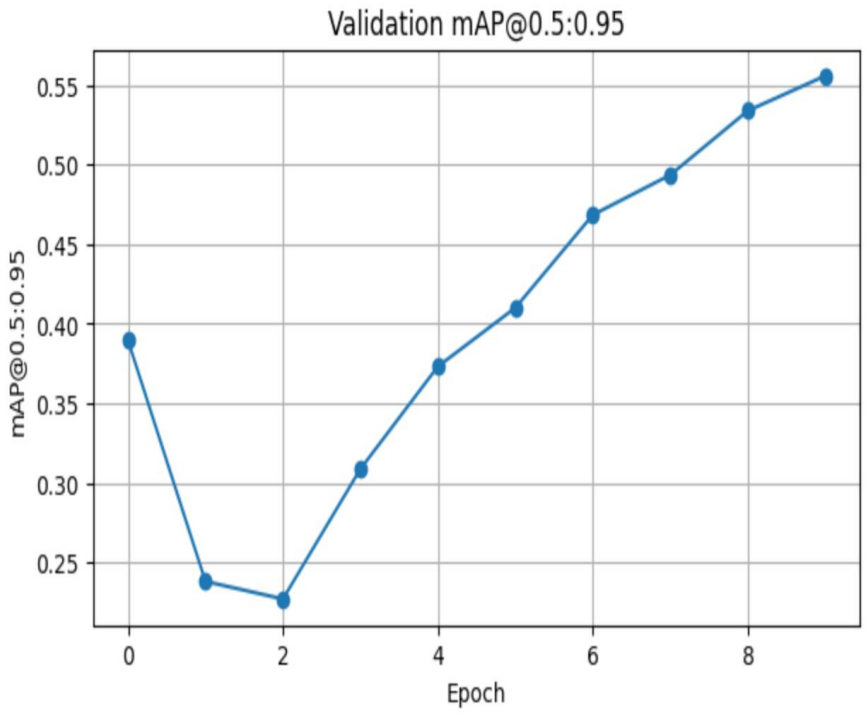
Đánh giá: Chỉ số này cho thấy mô hình đã nắm bắt rất tốt các đặc trưng của 4 loại phương tiện. Khả năng phát hiện đúng đối tượng (với ngưỡng IoU cơ bản 50%) là rất cao.



Đồ thị Đánh giá - Validation mAP@0.5:0.95

Xu hướng: Tương tự mAP@0.5, đồ thị vượt qua giai đoạn bất ổn ban đầu và tăng trưởng tuyến tính, đạt mức ~55% ở epoch cuối.

Đánh giá: Chỉ số khắt khe này tăng đều chứng tỏ chất lượng của các Bounding Box dự đoán ngày càng "ôm sát" đối tượng thực tế trên nhiều ngưỡng IoU khác nhau.



03

FASTER R-CNN

Phương pháp 2 — Faster R-CNN (Two-stage)

- Faster R-CNN là mô hình phát hiện đối tượng thuộc nhóm **two-stage detector**
- Gồm 2 bước chính:
  1. **Region Proposal Network (RPN)** – sinh vùng nghi ngờ
  2. **Detection Head** – phân loại & dự đoán bounding box
- Sử dụng backbone CNN (ResNet/VGG) để trích xuất đặc trưng
- Được fine-tune từ pretrained (ImageNet/COCO)

Ưu tiên độ chính xác cao hơn tốc độ => tốc độ chậm.

Kiến trúc Faster R-CNN

Ảnh đầu vào → Backbone (ResNet50 + FPN / VGG)

Feature map được đưa vào:

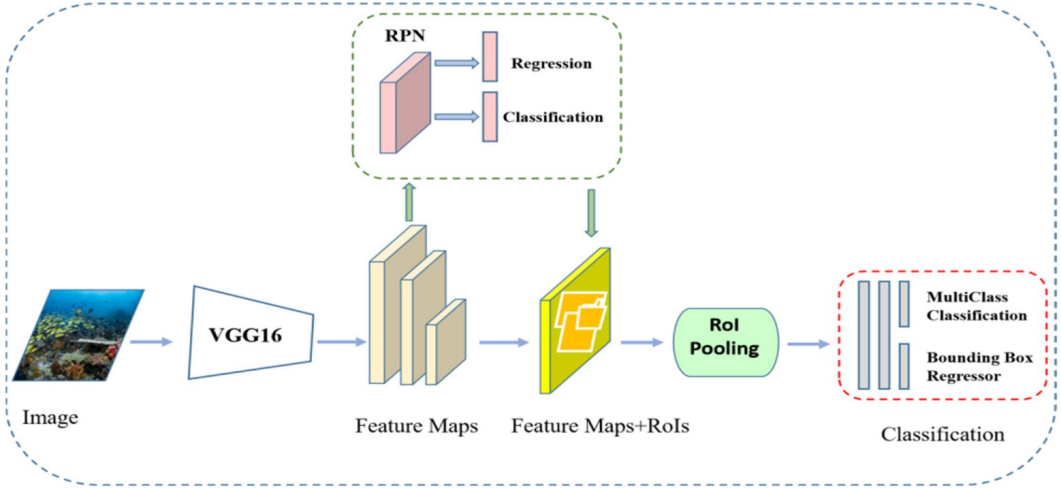
- RPN → sinh region proposals (anchor-based)

Áp dụng:

- RoI Pooling / RoI Align để chuẩn hóa kích thước vùng

Đầu ra:

- Classifier (phân loại object)
- Bounding Box Regressor (tinh chỉnh vị trí)



Cơ chế RPN và Proposal

RPN sử dụng anchor boxes với nhiều:

- Kích thước (scale)
- Tỉ lệ (aspect ratio)

Với mỗi anchor:

- Dự đoán objectness score
- Dự đoán offset bounding box

Sử dụng IoU để xác định positive/negative samples

Áp dụng Non-Maximum Suppression (NMS) để loại bỏ các box trùng lặp, giữ lại các vùng có khả năng chứa object cao.

Huấn luyện

Tổng quan bài toán

- Model: **Faster R-CNN (ResNet50 + FPN, pretrained COCO)**
- Dataset: **Pascal VOC 2007**

Task: Object Detection (4 class): car, bus, motorbike, bicycle

Training: fine-tune từ pretrained model

Dataset & Setup

Tổng dữ liệu:
- Train: 1034 images
- Val: 258 images
- Test: 1309 images

Filter:
- Chỉ giữ ảnh có vehicle → giảm từ ~5000 → ~1300 ảnh

Kết quả Training

Epoch progression:

- Epoch 1:
  - Loss: **0.3034**
  - mAP@0.5: **0.8202**
  - mAP@0.5:0.95: **0.4417**
- Epoch 2:
  - Loss: **0.1899**
  - mAP@0.5: **0.8606**
  - mAP@0.5:0.95: **0.5449**

Trend:

- Loss giảm nhanh
- mAP tăng rõ → model học tốt ngay từ đầu (do pretrained)

```
Epoch [1/10] Loss: 0.3034 | Val mAP@0.5: 0.8202 | Val mAP@0.5:0.95: 0.4417 |
Time: 6.80 min
Epoch [2/10] Loss: 0.1899 | Val mAP@0.5: 0.8606 | Val mAP@0.5:0.95: 0.5449 |
Time: 6.82 min
Epoch [3/10] Loss: 0.1527 | Val mAP@0.5: 0.8712 | Val mAP@0.5:0.95: 0.5603 |
Time: 6.81 min
Epoch [4/10] Loss: 0.1122 | Val mAP@0.5: 0.8765 | Val mAP@0.5:0.95: 0.5988 |
Time: 6.82 min
Epoch [5/10] Loss: 0.1021 | Val mAP@0.5: 0.8753 | Val mAP@0.5:0.95: 0.5993 |
Time: 8.13 min
Epoch [6/10] Loss: 0.0974 | Val mAP@0.5: 0.8680 | Val mAP@0.5:0.95: 0.5945 |
Time: 6.80 min
Epoch [7/10] Loss: 0.0926 | Val mAP@0.5: 0.8698 | Val mAP@0.5:0.95: 0.6005 |
Time: 6.86 min
Epoch [8/10] Loss: 0.0922 | Val mAP@0.5: 0.8699 | Val mAP@0.5:0.95: 0.6005 |
Time: 6.80 min
Epoch [9/10] Loss: 0.0916 | Val mAP@0.5: 0.8733 | Val mAP@0.5:0.95: 0.6004 |
Time: 6.84 min
Epoch [10/10] Loss: 0.0916 | Val mAP@0.5: 0.8732 | Val mAP@0.5:0.95: 0.6000 |
Time: 6.83 min
Best val mAP@0.5: 0.8764656782150269
```

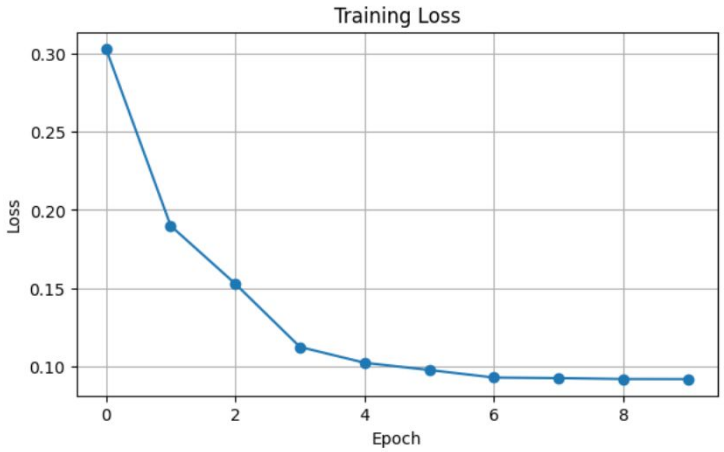
Kết quả Training

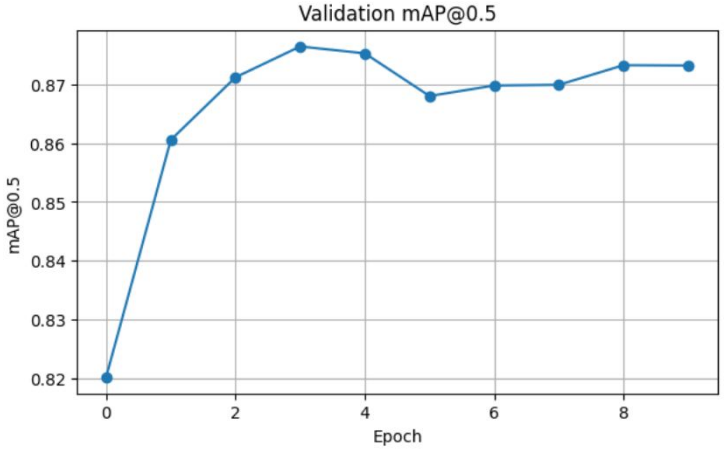
- Test mAP@0.5: 0.8623
- Test mAP@0.5:0.95: 0.5862

Test set: kept 1309 / 4952 images

Result:

- FPS: 7.93
- Seconds / image: 0.1262





Ưu điểm của Faster R-CNN

- Độ chính xác cao (high mAP) đặc biệt ở mAP@0.5 và localization tốt
- Phát hiện tốt vật thể nhỏ nhờ region proposals tập trung
- Ổn định trong môi trường phức tạp nhiều object, bị che khuất (occlusion)
- Transfer learning hiệu quả hoạt động tốt với dataset nhỏ

👉 Phù hợp:

- Server-side inference
- Bài toán yêu cầu độ chính xác cao

Nhược điểm và đánh giá

- Tốc độ chậm (~5–10 FPS) không phù hợp real-time
- Chi phí tính toán cao cần GPU mạnh
- Pipeline phức tạp, nhiều hyperparameters:
  - anchor size
  - NMS threshold
  - proposal count
- Khó deploy trên edge device

👉 Kết luận:

- Faster R-CNN đạt độ chính xác cao
- Nhưng không phù hợp hệ thống nhúng hoặc realtime, thích hợp cho **offline processing hoặc cloud AI**

04

SO SÁNH KẾT QUẢ

Kết quả thực nghiệm — Bảng so sánh

Bảng tóm tắt các chỉ số đánh giá thu được trên tập kiểm thử VOC2007 sau huấn luyện với cấu hình tiêu chuẩn

Model	mAP@0.5	mAP@0.5:0.95	FPS (inference)
YOLOv8 (medium)	0.7804	0.5603	198.77
Faster R-CNN (ResNet50)	0.8623	0.5862	7.93

Faster R-CNN đạt độ chính xác cao hơn (mAP), trong khi YOLOv8 có tốc độ suy luận (FPS) nhanh hơn vượt trội (gấp ~25 lần). Điều này thể hiện sự đánh đổi giữa độ chính xác và tốc độ.

Phân tích Trade-off: Speed vs Accuracy

Quan sát chính

YOLOv8: Tối ưu cho ứng dụng real-time. Tốc độ vượt trội (~198 FPS) nhưng vẫn giữ mAP ở mức hợp lý.

Faster R-CNN: Ưu thế tuyệt đối về độ chính xác (đặc biệt với vật thể nhỏ/bị che khuất), nhưng tốc độ suy luận chậm hơn đáng kể (~8 FPS).

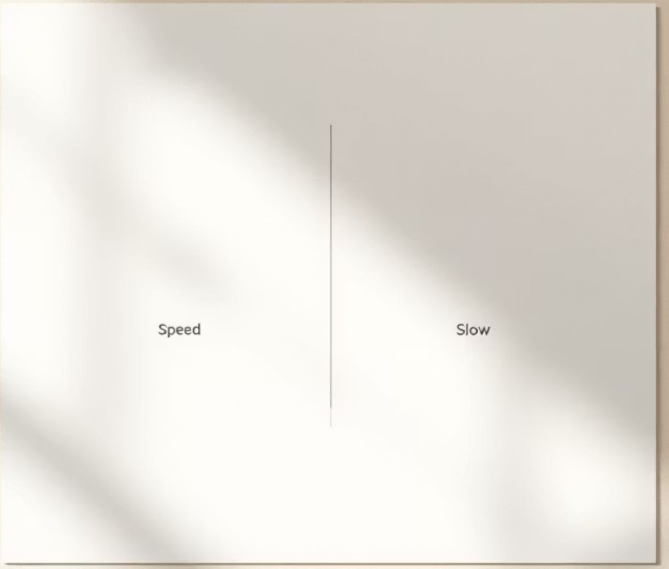
Ứng dụng phù hợp

YOLOv8: Phù hợp hệ thống giám sát giao thông thời gian thực, triển khai trên thiết bị Edge (IoT/Camera).

Faster R-CNN: Phù hợp phân tích video hậu kỳ, pháp y hình ảnh (forensic) yêu cầu độ tin cậy cao.

Chiến lược tối ưu

Kết hợp: dùng YOLOv8 để xử lý sơ bộ + Faster R-CNN cho những vùng nghi ngờ (cascade/ensemble) hoặc sử dụng model distillation để đạt trade-off tốt hơn.



Phần mở rộng: Phân tích lỗi định tính

False Positive (Nhận nhầm lớp/nền):

- Thường xảy ra với:
  - Vùng có **hình dạng/texture giống phương tiện**
  - **Phản xạ trên kính**, biển quảng cáo
- Nguyên nhân:
  - Model học **pattern hình dạng** thay vì ngữ cảnh
  - Dataset chưa đa dạng background

Miss Detection (Bỏ sót vật thể):

- Chủ yếu do:
  - **Small objects** (vật thể nhỏ)
  - **Severe occlusion** (bị che khuất)
- Nguyên nhân:
  - Resolution đầu vào còn hạn chế
  - Thiếu dữ liệu cho object nhỏ

Phần mở rộng: Giải thích mô hình & Hướng phát triển

Giải thích mô hình :

- **Hiện tại:**
  - Phân tích lỗi dựa trên: Quan sát bounding box, so sánh prediction vs ground truth (hiện tại nhóm đang thực hiện ở mức định tính).
- **Hạn chế:**
  - Chưa xác định chính xác vùng pixel ảnh hưởng đến quyết định của model (đây là "hộp đen" cần mở).

Hướng mở rộng (Future Work):

- **Phương pháp đề xuất:**
  - Áp dụng **Grad-CAM / Score-CAM** (Grad-CAM: Gradient-weighted Class Activation Mapping).
  - Mục tiêu: Trực quan hóa vùng ảnh đóng góp vào quyết định để xác định nguyên nhân FP/FN và định hướng cải tiến (ví dụ: tinh chỉnh Data Augmentation, áp dụng Focal Loss).

05

KẾT LUẬN

Kết luận & Hướng phát triển

Kết luận chính

- **Tốc độ & Thực thời:** YOLOv8 là lựa chọn tối ưu, đáp ứng xuất sắc cho hệ thống giám sát thời gian thực (đạt tốc độ ~198 FPS).
- **Độ chính xác & Chi tiết:** Faster R-CNN duy trì sức mạnh vượt trội khi xử lý vật thể nhỏ, phù hợp cho phân tích hậu kỳ đòi hỏi độ tin cậy cao.
- **Chiến lược:** Việc sử dụng kết hợp (Ensemble/Cascade) là giải pháp thực tiễn nhất để tận dụng ưu điểm của cả hai mô hình.

Hướng phát triển

- **Về Dữ liệu (Data):** Mở rộng tập dữ liệu với các bối cảnh khó (ảnh ban đêm, thời tiết xấu) và tối ưu pipeline đánh giá (reproducibility).
- **Về Mô hình (Model):** Thử nghiệm nén mô hình (Knowledge Distillation, Quantization) để đưa mô hình nặng xuống thiết bị Edge.
- **Về Thuật toán:** Nghiên cứu các kiến trúc Attention-based (như DETR) và Pretraining tự giám sát (Self-supervised) để tăng khả năng tổng quát hóa.

06

DEMO

Demo & Hướng triển khai

Bản mẫu tương tác (Prototype)

- **Công cụ:** Xây dựng giao diện bằng **Streamlit** hoặc **Gradio**.
- **Tính năng chính:** Upload ảnh/video, chuyển đổi qua lại giữa 2 model, tùy chỉnh Confidence Threshold.
- **Hiển thị trực quan:** Bounding Box, FPS inference, và công tắc bật/tắt **Grad-CAM** (Heatmap).
- **Môi trường:** Dễ dàng triển khai (deploy) lên GPU Cloud (AWS/GCP) để demo trực ti

Triển khai Production (Khuyến nghị)

- **Đóng gói (Containerization):** Sử dụng **Docker** để chuẩn hóa và cô lập môi trường chạy.
- **Tối ưu suy luận (Optimization):** Chuyển đổi trọng số (weights) sang định dạng **ONNX** hoặc **TensorRT** để tối đa hóa tốc độ inference trên GPU/Edge device.
- **Vận hành & Giám sát:** * Thiết lập cân bằng tải (Load balancing) trên Inference Server.

Q&A